

Pipes

A pipe transforms a value **right inside the template**, using the `|` symbol — without touching the actual value stored in your component. This guide covers every built-in pipe you'll use day to day, how to write your own, and where pipes fit in a real screen.

1 · THE PROBLEM

Raw data doesn't always look presentable

We have

```
price = 49.9;  
<p>{{ price }}</p>  
// renders as: 49.9
```

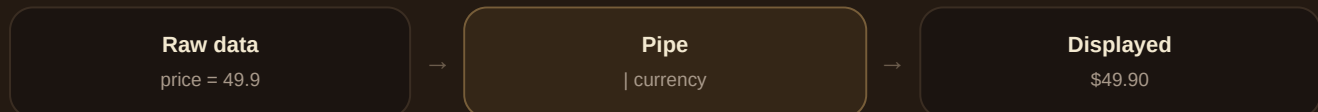
We want

```
price = 49.9;  
<p>{{ price | currency }}</p>  
// renders as: $49.90
```

We need to format data for humans — dates, prices, text case — without changing the underlying value stored in the component.

2 · WHAT IS A PIPE?

A one-way transform, applied in the template



WITHOUT A PIPE

You'd write formatting logic (`Intl.NumberFormat`, date parsing...) by hand inside every component that needs it

WITH A PIPE

One keyword in the template — Angular ships the common ones, and you can write your own

Why we need them: it keeps formatting logic out of your component class, reusable, and consistent across the whole app.

Formatting dates

```
today = new Date();
<p>{{ today | date:'longDate' }}</p>
// Result: August 24, 2026
```

The date pipe accepts a format string — 'short', 'medium', 'longDate', or a custom pattern like 'dd/MM/yyyy'.

FORMAT	OUTPUT
'short'	8/24/26, 12:00 PM
'medium'	Aug 24, 2026, 12:00:00 PM
'longDate'	August 24, 2026
'shortDate'	8/24/26

Real-world usage: order timestamps, "last updated" labels, event schedules.

Formatting prices

```
price = 49.9;
<p>{{ price | currency:'USD' }}</p>
// Result: $49.90
```

Angular formats the number with the right currency symbol, thousands separators, and two decimal places automatically.

BEFORE	AFTER
49.9	\$49.90
1250	\$1,250.00
3.5	\$3.50

Real-world usage: product prices, invoices, checkout totals, discounts.

Uppercase & lowercase

uppercase

```
<p>{{ userName | uppercase }}</p>
// 'Ahmed' -> AHMED
```

Used for badges, labels, section headers.

lowercase

```
<p>{{ email | lowercase }}</p>
// 'Ahmed' -> ahmed
```

Used to normalize emails & usernames.

Real-world usage: "NEW" badges, category tags, comparing emails without case-sensitivity bugs.

Niche but handy

PIPE	EXAMPLE	RESULT
percent	{{ 0.256 percent }}	26%
decimal	{{ 3.14159 number:'1.2-2' }}	3.14
json	{{ obj json }}	pretty-printed JSON — handy for debugging
slice	{{ items slice:0:3 }}	first 3 items of an array or string

7 · CUSTOM PIPES

When built-in pipes aren't enough

Problem: what if we need our own formatting rule, like shortening long text?

```
@Pipe({ name: 'truncate' })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit = 20) {
    return value.length > limit
      ? value.slice(0, limit) + '...'
      : value;
  }
}

{{ description | truncate:15 }}
// Result: "Angular is a po..."
```

A custom pipe is a class with a `transform()` method — reusable anywhere in the app. Real-world: product descriptions, comment previews, notification text, table cells with limited width.

8 · PIPES IN A REAL APP

One column of data, three different pipes, zero extra component code

```
<h3>{{ productName | uppercase }}</h3>
<p class="price">{{ price | currency }}</p>
<p class="date">Added {{ addedDate | date:'mediumDate' }}</p>

// Result: USB-C HUB · $34.50 · Added Aug 24, 2026
```

Notice the component never formats anything itself — `productName`, `price`, and `addedDate` stay plain, simple values. All the formatting lives in the template. This is why pipes matter in real apps: tables, lists, and cards all need consistent formatting, and pipes give it to you for free.